LAB WEEK 4

Implementation of a Normalized Database in MySQL

The Pressly Ad Agency - Advertising Database

Practical focus: Create → Populate → Verify → Query

Normalization theory is assumed from the lecture sessions.

Lab Objectives

- Create and select a database in MySQL.
- Create tables using suitable data types and integrity constraints.
- Define primary keys, composite keys, and foreign keys.
- Implement a database that has already been normalized to Third Normal Form (3NF).
- Insert sample data in the correct dependency order.
- Retrieve data using SELECT statements.
- Combine related tables using JOIN operations.
- Verify that the normalized tables can reconstruct the original information without losing data.

1. Database after Normalization to Third Normal Form

The normalization steps have been covered in the lectures. For this lab, use the following final 3NF relations.

AD CAMPAIGN

AdCampaignID	AdCampaignName	StartDate	Duration	CampaignMgrID
111	SummerFun20	6.6.2020	12 days	CM100
222	SummerZing20	6.8.2020	30 days	CM101
333	FallBall20	6.9.2020	12 days	CM102
444	AutmnStyle20	6.9.2020	5 days	CM103
555	AutmnColors20	6.9.2020	3 days	CM100

Primary Key: AdCampaignID

Foreign Key: CampaignMgrID

CAMPAIGN MANAGER

CampaignMgrID	CampaignMgrName
CM100	Roberta
CM101	Sue
CM102	John
CM103	Nancy

Primary Key: CampaignMgrID

MODUS

ModusID	Media	Range
1	TV	Local
2	TV	National
3	Radio	Local
4	Radio	National
5	Print	Local
6	Print	National
7	Internet	National

Primary Key: ModusID

AD CAMPAIGN MIX

AdCampaignID	ModusID	BudgetPctg
111	1	50%
111	2	50%
222	1	60%
222	3	30%
222	5	10%
333	3	80%
333	4	20%
444	6	100%
555	3	100%

Composite Primary Key: (AdCampaignID, ModusID)

Foreign Keys: AdCampaignID, ModusID

2. MySQL Implementation

You will now implement the normalized database in MySQL. The database will be called advertise.

PART A - Creating the Database

Step 1: Create and Select the Database

(establish connections – see *pages 2 and 3* of lab 1 manual *'working with MySQL Workbench '*)

Open MySQL Workbench, create a new SQL tab, and execute:

```
CREATE DATABASE advertise;

USE advertise;
```

Verify that the database has been created:

```
SHOW DATABASES;
```

PART B - Creating the Tables

Create the parent tables before the tables that contain foreign keys. Use the following order:

1. CampaignManager
2. Modus
3. AdCampaign
4. AdCampaignMix

Step 2: Create the CampaignManager Table

```
CREATE TABLE CampaignManager (
    CampaignMgrID VARCHAR(5) PRIMARY KEY,
    CampaignMgrName VARCHAR(50) NOT NULL
);
```

CampaignMgrID uniquely identifies each manager, so it is defined as the primary key.

Step 3: Create the Modus Table

```
CREATE TABLE Modus (
    ModusID INT PRIMARY KEY,
    Media VARCHAR(20) NOT NULL,
    `Range` VARCHAR(20) NOT NULL
);
```

ModusID uniquely identifies an advertising method. Backticks are used around Range because RANGE can be interpreted as a MySQL keyword.

Step 4: Create the AdCampaign Table

```
CREATE TABLE AdCampaign (
    AdCampaignID INT PRIMARY KEY,
    AdCampaignName VARCHAR(50) NOT NULL UNIQUE,
    StartDate DATE NOT NULL,
    Duration INT NOT NULL,
    CampaignMgrID VARCHAR(5) NOT NULL,

    FOREIGN KEY (CampaignMgrID)
        REFERENCES CampaignManager(CampaignMgrID)
);
```

Duration is stored as an integer (for example, 12 rather than "12 days") so that numeric comparisons and calculations can be performed.

Step 5: Create the AdCampaignMix Table

```
CREATE TABLE AdCampaignMix (
    AdCampaignID INT,
    ModusID INT,
    BudgetPctg DECIMAL(5,2) NOT NULL,

    PRIMARY KEY (AdCampaignID, ModusID),

    FOREIGN KEY (AdCampaignID)
        REFERENCES AdCampaign(AdCampaignID),

    FOREIGN KEY (ModusID)
        REFERENCES Modus(ModusID)
);
```

The pair (AdCampaignID, ModusID) forms a composite primary key because the same campaign can use several modes, and the same modus can be used by several campaigns.

PART C - Verifying the Database Structure

Step 6: Display the Tables

```
SHOW TABLES;
```

You should see four tables: AdCampaign, AdCampaignMix, CampaignManager, and Modus.

Step 7: Examine the Structure of Each Table

```
DESCRIBE CampaignManager;
DESCRIBE Modus;
DESCRIBE AdCampaign;
DESCRIBE AdCampaignMix;
```

Observe the column names, data types, key columns, and NULL/NOT NULL settings.

PART D - Inserting Sample Data

Insert data into the parent tables first, followed by the child tables, so that foreign-key references already exist.

Step 8: Insert Campaign Manager Data

```
INSERT INTO CampaignManager
(CampaignMgrID, CampaignMgrName)
VALUES
('CM100', 'Roberta'),
('CM101', 'Sue'),
('CM102', 'John'),
('CM103', 'Nancy');
```

```
SELECT * FROM CampaignManager;
```

Step 9: Insert Modus Data

```
INSERT INTO Modus
(ModusID, Media, `Range`)
VALUES
(1, 'TV', 'Local'),
(2, 'TV', 'National'),
(3, 'Radio', 'Local'),
(4, 'Radio', 'National'),
(5, 'Print', 'Local'),
(6, 'Print', 'National'),
(7, 'Internet', 'National');
```

```
SELECT * FROM Modus;
```

Step 10: Insert Advertising Campaign Data

```
INSERT INTO AdCampaign
(AdCampaignID, AdCampaignName, StartDate, Duration, CampaignMgrID)
VALUES
(111, 'SummerFun20', '2020-06-06', 12, 'CM100'),
(222, 'SummerZing20', '2020-08-06', 30, 'CM101'),
(333, 'FallBall20', '2020-09-06', 12, 'CM102'),
(444, 'AutmnStyle20', '2020-09-06', 5, 'CM103'),
(555, 'AutmnColors20', '2020-09-06', 3, 'CM100');
```

```
SELECT * FROM AdCampaign;
```

Step 11: Insert Ad Campaign Mix Data

```
INSERT INTO AdCampaignMix
(AdCampaignID, ModusID, BudgetPctg)
VALUES
(111, 1, 50),
(111, 2, 50),
(222, 1, 60),
(222, 3, 30),
(222, 5, 10),
(333, 3, 80),
(333, 4, 20),
(444, 6, 100),
(555, 3, 100);
```

```
SELECT * FROM AdCampaignMix;
```

PART E - Viewing and Joining the Data

Step 12: Display the Contents of All Tables

```
SELECT * FROM CampaignManager;
SELECT * FROM Modus;
SELECT * FROM AdCampaign;
SELECT * FROM AdCampaignMix;
```

Compare the output with the four normalized relations shown in Section 4.

Step 13: Display Campaigns with Their Managers

```
SELECT
    AdCampaign.AdCampaignID,
    AdCampaign.AdCampaignName,
    CampaignManager.CampaignMgrName
FROM AdCampaign
JOIN CampaignManager
    ON AdCampaign.CampaignMgrID = CampaignManager.CampaignMgrID;
```

Step 14: Display Campaigns with Their Advertising Modes

```
SELECT
    AdCampaign.AdCampaignName,
    Modus.Media,
    Modus.`Range`,
    AdCampaignMix.BudgetPctg
FROM AdCampaign
JOIN AdCampaignMix
    ON AdCampaign.AdCampaignID = AdCampaignMix.AdCampaignID
JOIN Modus
    ON AdCampaignMix.ModusID = Modus.ModusID;
```

A campaign may appear more than once because a campaign can use several advertising modes.

Step 15: Reconstruct the Original Relation

```
SELECT
    ac.AdCampaignID,
    ac.AdCampaignName,
    ac.StartDate,
    ac.Duration,
    ac.CampaignMgrID,
    cm.CampaignMgrName,
    m.ModusID,
    m.Media,
    m.`Range`,
    acm.BudgetPctg
FROM AdCampaign ac
JOIN CampaignManager cm
    ON ac.CampaignMgrID = cm.CampaignMgrID
JOIN AdCampaignMix acm
    ON ac.AdCampaignID = acm.AdCampaignID
JOIN Modus m
    ON acm.ModusID = m.ModusID
ORDER BY ac.AdCampaignID, m.ModusID;
```

3. Guided Practice Queries

Task 1 - Display campaign IDs and campaign names

```
SELECT AdCampaignID, AdCampaignName
FROM AdCampaign;
```

Task 2 - Display campaigns managed by CM100

```
SELECT *
FROM AdCampaign
WHERE CampaignMgrID = 'CM100';
```

Task 3 - Display all TV advertising modes

```
SELECT *
FROM Modus
WHERE Media = 'TV';
```

Task 4 - Display all National advertising modes

```
SELECT *
FROM Modus
WHERE `Range` = 'National';
```

Task 5 - Display campaigns lasting more than 10 days

```
SELECT *
FROM AdCampaign
WHERE Duration > 10;
```

Task 6 - Sort campaigns by start date

```
SELECT *
FROM AdCampaign
ORDER BY StartDate;
```

4. Testing Database Constraints

Purpose: These two tests demonstrate how MySQL enforces key constraints to protect data integrity.

Step 16: Test a Primary-Key Constraint

Try to insert another campaign manager using an ID that already exists:

```
INSERT INTO CampaignManager
VALUES ('CM100', 'Another Manager');
```

1. Did MySQL accept the record?

No. MySQL rejected the record because CM100 already exists as a primary key value.

2. What error message was generated?

A typical MySQL message is:
ERROR 1062 (23000): Duplicate entry 'CM100' for key 'PRIMARY'

The exact wording can vary slightly depending on the MySQL version or client.

3. Why can two campaign managers not have the same CampaignMgrID?

CampaignMgrID is the primary key of the CampaignManager table. Primary-key values must be unique, so two different records cannot use the same CampaignMgrID.

Key idea: Primary key → prevents duplicate identifiers.

Step 17: Test a Foreign-Key Constraint

Try to insert a campaign that references a manager who does not exist:

```
INSERT INTO AdCampaign
VALUES (777, 'TestCampaign', '2020-10-01', 10, 'CM999');
```

1. Did MySQL accept the record?

No. MySQL rejected the record because the value CM999 does not match any existing CampaignMgrID in the parent table.

2. Does CM999 exist in CampaignManager?

No. The CampaignManager table contains CM100, CM101, CM102 and CM103; CM999 is not present.

CampaignMgrID	CampaignMgrName
CM100	Roberta
CM101	Sue
CM102	John
CM103	Nancy

3. Why does the foreign-key constraint prevent the insertion?

CampaignMgrID in AdCampaign is a foreign key that references CampaignMgrID in CampaignManager. A foreign-key value must refer to an existing parent record. Since CM999 does not exist, inserting the campaign would create an invalid reference, so MySQL rejects it.

Typical MySQL message:

```
ERROR 1452 (23000): Cannot add or update a child row:
a foreign key constraint fails
```

Key idea: Foreign key -> prevents references to non-existent parent records and maintains referential integrity.

Summary

Constraint	What it protects
Primary Key	Prevents duplicate identifiers.
Foreign Key	Prevents invalid references to non-existent records.

5. More Practice Queries

Write and execute SQL statements for the following tasks.

1. Display the names of all campaign managers.
2. Display all campaigns managed by Roberta.
3. Display all advertising modes that use Radio.
4. Display the campaign name, media, range, and budget percentage for every campaign.
5. Display only campaigns that use a National advertising modus.
6. Display all advertising modes used by SummerZing20.
7. Display each campaign together with the name of its campaign manager.
8. Display campaign/modus combinations where more than 50% of the campaign budget is allocated to one modus.
9. Count the number of advertising modes used by each campaign. Hint: COUNT() and GROUP BY may be useful.
10. Calculate the total budget percentage allocated to each campaign. Hint: SUM() and GROUP BY may be useful.

Expected reasoning

A correctly allocated campaign should have a total BudgetPctg of 100 across all of its modes.